

1. Case-Based Learning vs Locally Weighted Regression

```
# =====
# CASE-BASED LEARNING vs LOCALLY WEIGHTED REGRESSION
# Recommendation System Example
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_error

# -----
# 1. CREATE SAMPLE USER DATASET
# -----

np.random.seed(42)

n = 150

data_usage = np.random.uniform(1, 50, n)
streaming_hours = np.random.uniform(1, 60, n)
calls = np.random.uniform(50, 500, n)

# Recommendation class
recommendation = []

for usage in data_usage:
    if usage < 15:
        recommendation.append("Basic")
    elif usage < 30:
        recommendation.append("Standard")
    else:
        recommendation.append("Premium")

# Rating for Locally Weighted Regression
rating = (
    2
    + 0.03 * data_usage
    + 0.02 * streaming_hours
    + np.random.normal(0, 0.4, n)
)

rating = np.clip(rating, 1, 5)

df = pd.DataFrame({
    "Data_Usage": data_usage,
    "Streaming_Hours": streaming_hours,
    "Calls": calls,
    "Recommendation": recommendation,
    "Rating": rating
})

print("Sample Dataset:")
print(df.head())

# -----
# 2. PREPARE FEATURES
# -----

X = df[[
    "Data_Usage",
    "Streaming_Hours",
    "Calls"
]].values

y_class = df["Recommendation"].values
y_rating = df["Rating"].values

# Normalize features
scaler = StandardScaler()
```

```

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Split data
X_train, X_test, y_class_train, y_class_test, \
y_rating_train, y_rating_test = train_test_split(
    X_scaled,
    y_class,
    y_rating,
    test_size=0.25,
    random_state=42
)

# =====
# 3. CASE-BASED LEARNING USING KNN
# =====

k = 5

cbl_model = KNeighborsClassifier(
    n_neighbors=k,
    metric="euclidean"
)

cbl_model.fit(
    X_train,
    y_class_train
)

class_predictions = cbl_model.predict(X_test)

accuracy = accuracy_score(
    y_class_test,
    class_predictions
)

print("\n=====")
print("CASE-BASED LEARNING RESULTS")
print("=====")

print("Accuracy:", round(accuracy * 100, 2), "%")

print("\nSample Recommendations:")

for i in range(10):
    print(
        "Actual:",
        y_class_test[i],
        "| Predicted:",
        class_predictions[i]
    )

# =====
# 4. LOCALLY WEIGHTED REGRESSION FUNCTION
# =====

def locally_weighted_regression(
    X_train,
    y_train,
    X_query,
    tau=1.0
):
    predictions = []

    # Add bias column
    X_train_bias = np.c_[
        np.ones(len(X_train)),
        X_train
    ]

    for x in X_query:
        # Add bias
        x_bias = np.r_[1, x]

```

```

        # Calculate distance
        distances = np.sum(
            (X_train - x) ** 2,
            axis=1
        )

        # Gaussian weights
        weights = np.exp(
            -distances / (2 * tau ** 2)
        )

        # Weight matrix
        W = np.diag(weights)

        # Calculate regression parameters
        theta = np.linalg.pinv(
            X_train_bias.T @ W @ X_train_bias
        ) @ (
            X_train_bias.T @ W @ y_train
        )

        # Predict rating
        prediction = x_bias @ theta

        predictions.append(prediction)

    return np.array(predictions)

# =====
# 5. LOCALLY WEIGHTED REGRESSION PREDICTION
# =====

tau = 1.0

rating_predictions = locally_weighted_regression(
    X_train,
    y_rating_train,
    X_test,
    tau
)

mae = mean_absolute_error(
    y_rating_test,
    rating_predictions
)

rmse = np.sqrt(
    mean_squared_error(
        y_rating_test,
        rating_predictions
    )
)

print("\n=====")
print("LOCALLY WEIGHTED REGRESSION RESULTS")
print("=====")

print("MAE :", round(mae, 3))
print("RMSE:", round(rmse, 3))

print("\nSample Rating Predictions:")

for i in range(10):
    print(
        "Actual:",
        round(y_rating_test[i], 2),
        "| Predicted:",
        round(rating_predictions[i], 2)
    )

# =====
# 6. ADD NEW DATA TO SHOW ADAPTABILITY
# =====

print("\n=====")
print("ADDING NEW USER DATA")

```

```

print("=====")

new_user = np.array([
    [40, 50, 300]
])

new_user_scaled = scaler.transform(new_user)

# Case-Based Learning Recommendation
new_recommendation = cbl_model.predict(
    new_user_scaled
)

# LWR Rating Prediction
new_rating = locally_weighted_regression(
    X_train,
    y_rating_train,
    new_user_scaled,
    tau
)

print("New User Details:")
print("Data Usage:", new_user[0][0], "GB")
print("Streaming Hours:", new_user[0][1])
print("Calls:", new_user[0][2])

print("\nCBL Recommended Plan:",
      new_recommendation[0])

print("LWR Predicted Rating:",
      round(new_rating[0], 2))

```

Sample Dataset:

	Data_Usage	Streaming_Hours	Calls	Recommendation	Rating
0	19.352466	54.587687	73.256775	Standard	3.796691
1	47.585001	15.134152	289.109584	Premium	4.320376
2	36.867703	9.548797	293.285805	Premium	3.640071
3	30.334266	29.877713	336.843456	Premium	3.443607
4	8.644913	59.153377	376.741100	Basic	3.434808

=====
CASE-BASED LEARNING RESULTS
=====

Accuracy: 92.11 %

Sample Recommendations:

Actual: Premium | Predicted: Premium
Actual: Standard | Predicted: Standard
Actual: Premium | Predicted: Premium
Actual: Standard | Predicted: Standard
Actual: Premium | Predicted: Premium
Actual: Basic | Predicted: Basic
Actual: Basic | Predicted: Basic
Actual: Basic | Predicted: Basic
Actual: Basic | Predicted: Basic
Actual: Standard | Predicted: Standard

=====
LOCALLY WEIGHTED REGRESSION RESULTS
=====

MAE : 0.341
RMSE: 0.421

Sample Rating Predictions:

Actual: 4.07 | Predicted: 3.62
Actual: 2.77 | Predicted: 2.56
Actual: 3.57 | Predicted: 3.71
Actual: 4.05 | Predicted: 3.63
Actual: 3.95 | Predicted: 4.1
Actual: 2.19 | Predicted: 2.58
Actual: 3.44 | Predicted: 3.41
Actual: 3.06 | Predicted: 2.57
Actual: 3.4 | Predicted: 2.88
Actual: 3.11 | Predicted: 3.26

=====
ADDING NEW USER DATA
=====

New User Details:
Data Usage: 40 GB
Streaming Hours: 50
Calls: 300

2. Locally Weighted Regression vs RBF Network

```
# =====
# LOCALLY WEIGHTED REGRESSION vs RBF NETWORK
# Function Approximation
# =====

import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.metrics import mean_squared_error

# -----
# 1. CREATE NONLINEAR DATASET
# -----

np.random.seed(42)

# Training data
X_train = np.linspace(
    0,
    10,
    100
).reshape(-1, 1)

# Nonlinear function with noise
y_train = (
    np.sin(X_train).ravel()
    + np.random.normal(0, 0.15, 100)
)

# Test data
X_test = np.linspace(
    0,
    10,
    200
).reshape(-1, 1)

# True function
y_true = np.sin(X_test).ravel()

# =====
# 2. LOCALLY WEIGHTED REGRESSION
# =====

def lwr(
    X_train,
    y_train,
    X_query,
    tau=0.5
):
    predictions = []

    # Add bias column
    X_train_bias = np.c_[
        np.ones(len(X_train)),
        X_train
    ]

    for x in X_query:
        # Distance from current point
        distances = np.sum(
            (X_train - x) ** 2,
            axis=1
        )

        # Gaussian weights
        weights = np.exp(
            -distances / (2 * tau ** 2)
```

```

    )

    # Create weight matrix
    W = np.diag(weights)

    # Calculate local parameters
    theta = np.linalg.pinv(
        X_train_bias.T @ W @ X_train_bias
    ) @ (
        X_train_bias.T @ W @ y_train
    )

    # Add bias
    x_bias = np.r_[1, x]

    # Prediction
    prediction = x_bias @ theta

    predictions.append(prediction)

    return np.array(predictions)

# Predict using LWR
tau = 0.5

y_lwr = lwr(
    X_train,
    y_train,
    X_test,
    tau
)

# =====
# 3. RBF NETWORK
# =====

# Number of RBF neurons
num_centers = 15

# Find RBF centers using K-Means
kmeans = KMeans(
    n_clusters=num_centers,
    random_state=42,
    n_init=10
)

kmeans.fit(X_train)

centers = kmeans.cluster_centers_

# -----
# Gaussian RBF Function
# -----

def rbf_features(
    X,
    centers,
    sigma
):
    features = []

    for center in centers:

        # Squared distance
        distance = np.sum(
            (X - center) ** 2,
            axis=1
        )

        # Gaussian RBF
        feature = np.exp(
            -distance / (2 * sigma ** 2)
        )

        features.append(feature)

```

```

    return np.column_stack(features)

# RBF width
sigma = 0.7

# Create hidden layer features
Phi_train = rbf_features(
    X_train,
    centers,
    sigma
)

Phi_test = rbf_features(
    X_test,
    centers,
    sigma
)

# Add bias column
Phi_train_bias = np.c_[
    np.ones(len(Phi_train)),
    Phi_train
]

Phi_test_bias = np.c_[
    np.ones(len(Phi_test)),
    Phi_test
]

# Train output weights using least squares
weights = np.linalg.pinv(
    Phi_train_bias
) @ y_train

# RBF predictions
y_rbf = Phi_test_bias @ weights

# =====
# 4. EVALUATE MODELS
# =====

mse_lwr = mean_squared_error(
    y_true,
    y_lwr
)

mse_rbf = mean_squared_error(
    y_true,
    y_rbf
)

print("=====")
print("MODEL COMPARISON")
print("=====")

print("LWR Mean Squared Error:",
      round(mse_lwr, 4))

print("RBF Mean Squared Error:",
      round(mse_rbf, 4))

# =====
# 5. VISUALIZE RESULTS
# =====

plt.figure(figsize=(10, 6))

plt.scatter(
    X_train,
    y_train,
    label="Training Data"
)

plt.plot(
    X_test,
    y_true,

```

```

        linewidth=2,
        label="True Function"
    )

plt.plot(
    X_test,
    y_lwr,
    linewidth=2,
    label="LWR Prediction"
)

plt.plot(
    X_test,
    y_rbf,
    linewidth=2,
    label="RBF Prediction"
)

plt.xlabel("Input X")
plt.ylabel("Output Y")

plt.title(
    "Function Approximation: LWR vs RBF Network"
)

plt.legend()
plt.grid(True)

plt.show()

# =====
# 6. DISPLAY RBF CENTERS
# =====

print("\nRBF Centers:")

print(
    np.sort(
        centers.flatten()
    )
)

```


=====

MODEL COMPARISON

=====

LWR Mean Squared Error: 0.0084

RBF Mean Squared Error: 0.0028

Function Approximation: LWR vs RBF Network

